

마구공학의 실체와 실천

마구(Harness)공학(Engineering)과 TypeScript 프로젝트
이야기

발표자 소개

이름: 맹주영

- 현재 TypeScript 주로 사용 (~~HKT같은 타입빌런 절대아님~~)
- 웹 서비스 주로 하다가 지금은 게임 관련 프로젝트 진행 중
- 잡부 (웹? 아닌 듯. 풀스택? pull stack. 개발? 코드 안 짤)

TMI:

- 게임, 뭐든 만들기, 게임 만들기 좋아했는데 살다보니 잡부
- Rust/그래픽스 등 기술 연구 재밌어함 (사이드 프로젝트)
- 재미/효율 추구, 구조/환경 중요하게 생각, ...

목차

1. AI 발전에 따른 개발 트렌드

- 지시공학?
- 요구사항 주도 개발?
- 마구공학?

2. 하네스 엔지니어링 실제 적용 후기

- 프로젝트 소개 ~~(이 세션의 거의 유일한 TypeScript 애기)~~
- 적용한 장치
- 느낀 점

3. 하네스 엔지니어링 더 훑어보기

- 다른 공개 사례
- 목표/산출물/제 조사 결과 및 의견 요약 등
- 저도 몰라요 살려주세요

AI 발전에 따른 개발 트렌드

1. 프롬프트 엔지니어링 (~~지식공학~~)
2. 스펙 주도 개발 (~~요구사항 주도 개발~~)
3. 하네스 엔지니어링 (~~마구공학~~)

프롬프트 엔지니어링

| 컨텍스트 엔지니어링 | 기타등등 |

| - | - |

| 입력 설계 | 지시 명확화 |

| 컨텍스트 주입 | 추론 유도 |

프롬프트 엔지니어링

| 컨텍스트 엔지니어링 | 기타등등 |

| - | - |

| 입력 설계 | 지시 명확화 |

| 컨텍스트 주입 | **추론 유도** |

당시 유행하던 추론 유도 기법들: Few-shot, CoT, ToT, Role/Persona Prompting

프롬프트 엔지니어링

| 컨텍스트 엔지니어링 | ~~기타등등~~ |

| - | - |

| 입력 설계 | ~~자시 명확화~~ |

| 컨텍스트 주입 | ~~추론 유도~~ |

프롬프트 엔지니어링

| 컨텍스트 엔지니어링 | ~~기타등등~~ |

| - | - |

| 입력 설계 | ~~지시 명확화~~ |

| 컨텍스트 주입 | ~~추론 유도~~ |

스펙 주도 개발

Spec Driven Development

사람은 스펙 위주로 작성, 구현은 AI

개발자의 역할 이동: 구현 -> 시스템 설계, 스펙 작성

주요 차이점: 구현이 아니라 스펙부터 작업, 구현 작업 전에 문제 정의를 명확히
만들

스펙 주도 개발

Requirements

Spec Driven Development

Design

사람은 스펙 위주로 작성, 구현은...

개발자의 역할 이동: 구현 -> 시스템 설계, 시스템 구현

Implementation

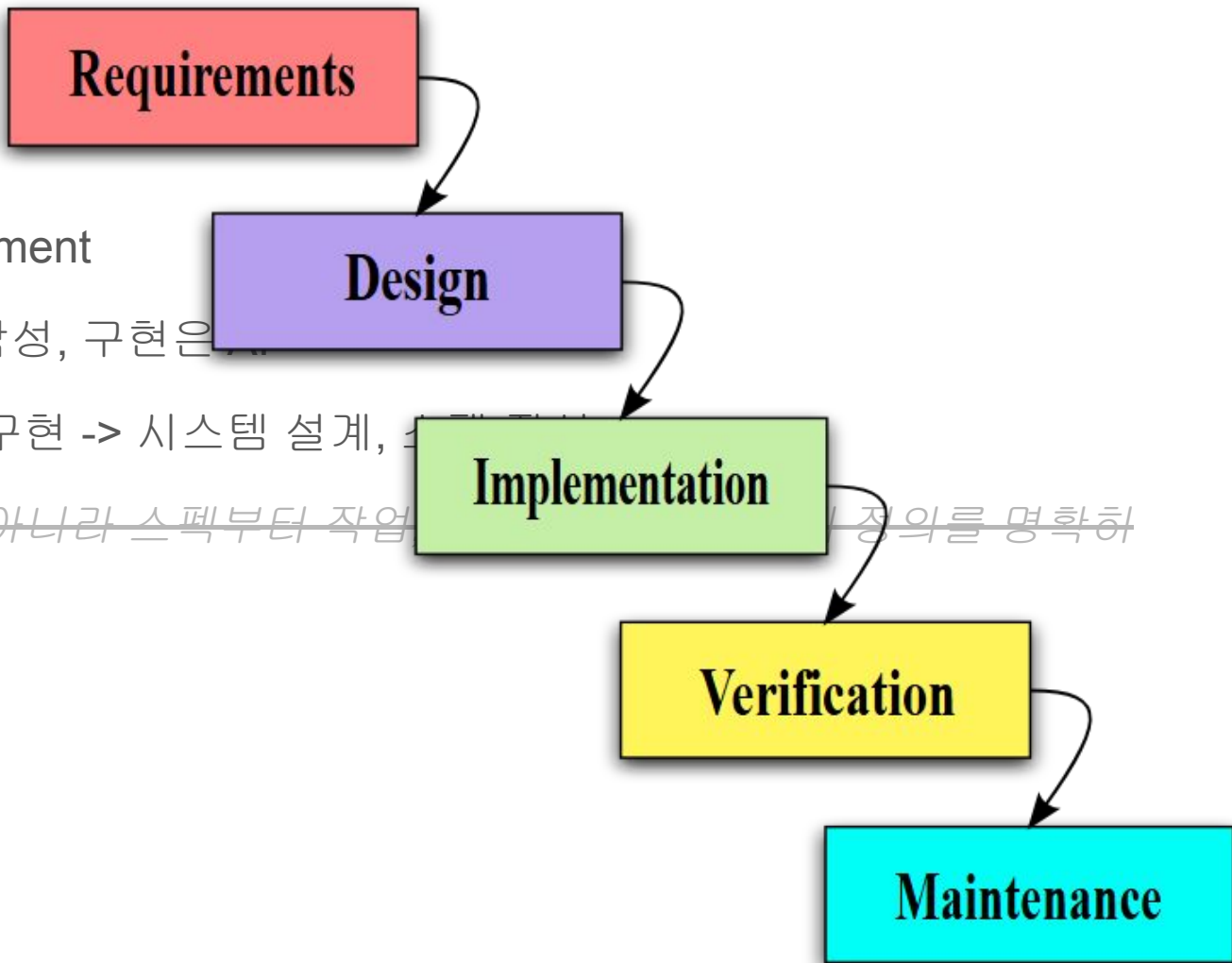
주요 차이점: 구현이 아니라 스펙부터 작업

정의를 명확히

만들

Verification

Maintenance



스펙 주도 개발

Spec Driven Development

사람은 스펙 위주로 작성, 구현은 AI

개발자의 역할 이동: 구현 -> 시스템 설계, 스펙 작성

~~주요 차이점: 구현이 아니라 스펙부터 작업, 구현 작업 전에 문제 정의를 명확히
만들~~

진짜 달라진 점: 구현 비용이 낮아짐에 따라 스펙에 집중

하네스 엔지니어링

좁은 정의: 에이전트 런타임과 루프

가이드: 좋은 기본 경로 안내

중간 정의: 에이전트가 잘 일하도록 하는 가이드와 센서

센서: 잘못된 걸 감지하는 피드백

넓은 정의: 지식/피드백/제어 체계 등 환경 전체

추상적 정의: 에이전트가 안 좋은 걸 반복하면, 그 실수를 어렵게 만드는 운영 습관?

하네스 엔지니어링 - 예시: 가상의 프로젝트

- C가 메인, CMake 사용, 근데 TypeScript도 있어서 npm도 씀
- 빌드 방법: **“npm run build”**
- 인간 에이전트: 빌드 방법 처음 한 번만 안내해도 실수 없이 잘 함
 - 물론 처음부터 너무 많은 걸 알려주면 인간 에이전트도 실수할 수 있음

하네스 엔지니어링 - 예시: 가상의 프로젝트

- C가 메인, CMake 사용, 근데 TypeScript도 있어서 npm도 씀
- 그리고 코딩 컨벤션, 여러가지 규칙 등 중요한 정보들, **암튼 뭐 많음**
- 인간 에이전트: 빌드 방법 처음 한 번만 안내해도 실수 없이 잘 함
- AI 에이전트: 매번 새롭고 습관도 없는데 당장 알아야 될 정보는 많음

```
$ cmake -B build
```

```
Error: use "npm run build" instead
```

하네스 엔지니어링 - 예시: 가상의 프로젝트

- 사람한테는 효용 없던 것도, **AI** 에이전트에게는 효용 있을 수 있음
- 인간 에이전트만이 아니라, 주로 **AI** 에이전트를 위한 환경 개선
- 인간 에이전트: ~~(어차피 저런 실수 안 하는데 저게 왜 필요함?)~~
- **AI** 에이전트: 동작 방식 차이에 따라 실수 유형 등이 다름

\$ cmake -B build

Error: use “npm run build” instead

AI 발전에 따른 개발 트렌드 요약

1. (호랑이 담배 피우던 시절)

- 코드를, AI 없이 사람이 직접 입력했음
- AI 자동완성은 허들 낮은 snippet 수준

2. 프롬프트 엔지니어링?

- AI는 실제로 생산성에 도움 되는 도구
- 잘 쓰려면 컨텍스트 엔지니어링 중요함

3. 스펙 주도 개발

- 스펙만 충분하면 토큰만 부어도 구현됨
- 코드보다는 진짜 가치 있는 스펙에 집중

4. 하네스 엔지니어링

- ~~○ 원래 하던 일인 것 같긴 한데 아무튼~~
- 개발환경은 사람보다 AI 에이전트 우선

하네스 엔지니어링 실제 적용 후기

1. 프로젝트 초간단 소개
2. 적용한 장치
3. 깨달은 점

프로젝트 초간단 소개

- GTA V 모딩 플랫폼 **FiveM** 기반의 커뮤니티 서버 개발 프로젝트
- 온라인에 공개된 코드는 대부분 **Lua** 기반, 품질 매우 낮음
- 그 **Lua** 기반 작업 당시 버그 많고 수정 비용 크고 생산성 낮았음
- 정상적인 구조로 기반부터 새로 만드는 프로젝트! 이벤트 소싱, DDD 등
- 언어는 **TypeScript** 채택 (선택지가 **Lua/C#/JavaScript**였음)
- 런타임별 **FiveM** 전용 API 있음: 20Hz Node.js server, V8 client, CEF UI

프로젝트에 적용한 장치들

- oh-my-openagent 사용하기
- 가짜 node/pnpm/npm 만들기
- just로 많은 행동 대응하기
- 파악하기 좋게 만들기
- AGENTS.md
- Claude Code를 위한 MCP
- 등

프로젝트에 적용한 장치: 가짜 node/pnpm/npm 만들기

- 가상의 프로젝트 예시의 가짜 cmake...의 TypeScript 버전
- 이 프로젝트는 **build/test/format/autofix** 등 일상 명령어를 **just**를 통해 호출
 - 예: pnpm run build 대신 just build
- **just build** 쓰면 되는데, 자꾸 에이전트가 잘못된 방법을 시도함
 - 예: npm run build, node --experimental-strip-types scripts/build.ts 등
- 물론 애초에 안 쓰게 하는 게 더 좋음, **escape hatch**는 필요 등 디테일 생략

프로젝트에 적용한 장치: just로 많은 행동 대응하기

- 호출 방식, 입력/출력 형식 등 동작이 일관된 기본 명령어 제공
- 이 프로젝트는 **build/test/format/autofix** 등 일상 명령어를 **just**를 통해 호출
 - 실패 시 자동 복구나 캐시 등 처리도 있고 다음 행동을 유도
- `just lint lib.ts` 실패. 다음 작업은? `npx eslint --fix lib.ts`? `just autofix lib.ts`?
 - Try “just autofix” first, or “just show-lint-error <path>” for details. Files with error: lib.ts
- `just autofix lib.ts`
 - No files changed. “just lint-error <path>” for details. Files with remaining errors: lib.ts
 - Success. No files with remaining errors.
 - Success. No files with remaining errors. Some files were changed but not re-staged.

프로젝트에 적용한 장치: 파악하기 좋게 만들기

- 폴더별 **overview.md** 작성해 역할과 책임, 주요 파일 등 문서화
- **overview.md** 등을 활용해 에이전트가 프로젝트를 파악하기에 좋은 툴 만들기
 - 대략적인 구조 파악: `just overview --summary path/to/directory`
 - 리뷰용 자세한 내용: `just overview --detail-for=review path/to/file`
- 진짜 이 정도로 단순하진 않았지만, 아무튼 초 허접한 툴인데 실제로 유용했음
- 이거 없었으면? 파일 목록 보고 이름으로 추측하고 모르면 파일 내용도 보고...

프로젝트에 적용한 장치: AGENTS.md

AGENTS.md 규칙:

- 환경/역할 등 상관 없이 무조건 읽게 하는 역할, 최소한의 중요 정보만 적음
- 구체적인 정보를 적기보다는, 구체적인 정보를 찾을 수 있게 하는 지도 역할

이 프로젝트의 AGENTS.md 내용:

- 프로젝트 초초초초초간단 소개 (목적, 상태 등)
- 주요 툴 사용법 예시를 포함한 툴 발견법
 - just overview와 just help를 포함한 몇 가지 예시, 그리고 툴 필요하면 just help 쓰라는 안내
- ~~에이전트 런타임 견드릴 시간 없어서 그냥 적어 둔~~ 진짜 중요한 일부 정책들

프로젝트에 적용한 장치: Claude Code를 위한 MCP

- Claude Code 시스템 프롬프트: “Bash 툴 사용 지양할 것”
- AGENTS.md: “탐색에 just overview 적극 사용할 것” (Bash 필요)
- Bash(just overview --summary src): 시스템 프롬프트에서 지양하랬는데...
- JustMCP(overview --summary src): (͡° ͜ʖ ͡°)

프로젝트에 적용한 장치들 요약

- **oh-my-openagent** 사용하기 (전 아직 안 써 봤는데 *oh-my-codex*가 더 좋대요)
- 일상 행동을 좋은 툴로 많이 커버하고 다음 행동 유도, 다른 길은 어렵게 만들기
- 에이전트가 프로젝트를 파악하기 좋게 만들기
 - 환경이나 파일에 대한 정보 뿐 아니라 쓸 수 있는 툴 같은 것들에 대한 정보도 포함
 - AGENTS.md에서는 방법만 안내

... 그리고 사소한 문제 해결 (Claude Code를 위한 just용 MCP workaround 등)

후기: 적용해보고 느낀 진짜 중요한 것들

1. 일단 AI 쓰기, AI 잘 쓰는 거 진짜 중요한 듯

모르겠으면 일단 무지성 oh-my-codex 짹먹!

후기: 적용해보고 느낀 진짜 중요한 것들

1. 일단 AI 쓰기, AI 잘 쓰는 거 진짜 중요한 듯
2. 의도를 명확히 하기, 안 그러면 에이전트가 우회함

(X) ~하면 안 됩니다, ~ 하지 마세요

(O) ~때문에 ~하면 안 됩니다, 대신 ~ 하세요, 잘 모르겠으면 자세한 내용은 ~
보세요

후기: 적용해보고 느낀 진짜 중요한 것들

1. 일단 AI 쓰기, AI 잘 쓰는 거 진짜 중요한 듯
2. 의도를 명확히 하기, 안 그러면 에이전트가 우회함
3. 문서 관리 진짜 잘 하기, 문서 충돌의 대가는 정말 큼

- SPEC.md: 이렇게 구현하면 됩니다.
- FEATURES.md: 저런 기능을 만들 거예요!
- 문서 둘 중 하나만 본 에이전트: 구현이 잘못됐네

후기: 적용해보고 느낀 진짜 중요한 것들

1. 일단 AI 쓰기, AI 잘 쓰는 거 진짜 중요한 듯
2. 의도를 명확히 하기, 안 그러면 에이전트가 우회함
3. 문서 관리 진짜 잘 하기, 문서 충돌의 대가는 정말 큼
4. 많이 검증하기, 그리고 가능하면 웬만하면 결정적으로
 - 이 파일 수정했네요? 그럼 관련 있을 것 같은 이 문서도 봐 주세요.
 - 이 문서에서 가리키는 저 경로에는 실제 **tracked** 파일이 없는데요?
 - 아 read/write가 경로가 아니라고요? 그럼 read, write로 써 주세요.
 - non-ascii 문자가 있네요. 꼭 필요한거면 `<!-- @non-ascii reason: ... -->` 붙여주세요.
 - `__test__`, `package.json`, `tsconfig.json`은 쓸데없으니 빼 주세요.

후기: 적용해보고 느낀 진짜 중요한 것들

1. 일단 AI 쓰기, AI 잘 쓰는 거 진짜 중요한 듯
2. 의도를 명확히 하기, 안 그러면 에이전트가 우회함
3. 문서 관리 진짜 잘 하기, 문서 충돌의 대가는 정말 큼
4. 많이 검증하기, 그리고 가능하면 웬만하면 결정적으로
5. 처음부터 구조 잘 잡기, 그리고 역시 스펙이 중요한 듯

하네스 엔지니어링 더 훑어보기

1. 하네스 엔지니어링의 다른 공개 사례
2. 하네스 엔지니어링의 목표
3. 하네스 엔지니어링의 산출물
4. 하네스 엔지니어링의 세부 주제
5. TMI 등

하네스 엔지니어링의 다른 공개 사례

OpenAI, Anthropic, AutoBE 등 여러 공개 사례 기반 요약

- **repo** 전체를 에이전트가 읽기 좋은 지식 시스템으로 만들기
- 취향이나 아키텍처 등을 권고 문서가 아니라 결정적 검사로 승격
- 일반 문서와 달리 강제 **hook**과 **always-on**/지도 문서는 분리해서 다르게 취급
- 하네스 잘 만들기 좋은 코드베이스를 의도적으로 만들기
- 큰 작업을 위한 **handoff**, 계획/구현 분리, 구조화된 계약, 서브에이전트
- 싼 결정적 검사는 앞에, 비싼 추론적 검사는 뒤에 배치 등 기타

하네스 엔지니어링의 목표

- 에이전트를 좋은 경로로 유도함
 - 좋은 경로는 프로젝트 표면이라 발견 쉬움, 나쁜 경로는 쓰기도 어렵고 나쁜 경로라는 게 금방 걸림
 - 실패를 구조화된 복구로 연결, 실패를 포함한 여러 상황을 예외가 아니라 설계된 흐름으로 취급
- 적절한 모든 지식을 에이전트가 읽을 수 있음
- 컨텍스트는 상황에 맞게 요약/지도 번들로 제공됨
- 표면별 권한이 잘 나뉘어 있음
- 반복 실수를 시스템에 흡수함
- 세션이 길어져도 **handoff**로 이어서 작업할 수 있음
- 사람이 모든 액션을 승인하지 않아도 통제가 유지됨, 지식의 상태/충돌 등 관리
- 런타임 고장은 자동으로 고쳐짐, 병렬 **workflow**가 자연스럽게 지원됨 등등

하네스 엔지니어링의 산출물

- 안내/지식: 지식 검색/관리 시스템
- AGENTS.md나 동급의 인덱스, 파일 하나일 필요는 없음 (환경/에이전트별 등)
- 정보 문서: 스펙, API 계약, 아키텍처 등에 대한 공식 기준
- 검색, 컨텍스트 번들: 관련 있는 내용 요약 + 자세한 내용 얻는 방법
- 관리: 지식 수정/반영 절차, 모순 감지 등

하네스 엔지니어링의 산출물

- 안내/지식: 지식 검색/관리 시스템
 - 실행/도구: 툴, 구조화된 런타임 계약
-
- 계약: **API**처럼, 에이전트 <-> 런타임, 에이전트 <-> 에이전트 소통 규칙 등
 - 객체: **tool call/result, approval/escalation request, handoff package** 등
 - 실패: 정책상 금지 (대안은?), 일부만 성공 (뭐가 성공?), 거부됨 (왜?) 등

하네스 엔지니어링의 산출물

- 안내/지식: 지식 검색/관리 시스템
 - 실행/도구: 툴, 구조화된 런타임 계약
 - 안전/통제: **sandbox**, **secret** 관리 등
-
- **sandbox**: DinD + cgroup 리소스 제한으로 어느 정도 날먹 가능할지도?
 - **secret**: 별도 컨테이너로 환경 분리하고 구조화된 런타임 계약으로 통신?

하네스 엔지니어링의 산출물

- 안내/지식: 지식 검색/관리 시스템
- 실행/도구: 툴, 구조화된 런타임 계약
- 안전/통제: **sandbox, secret** 관리 등
- 관측: 재현, 디버깅, 환경 개선 등을 위함
- 검증: 일 잘 했는지 판정하는 기준 및 테스트 (**acceptance**)
- 지속성: **handoff, progress** 등 아티팩트 및 그에 기반한 업무 프로세스 등 정의
- 정책: 권한 종류, 에이전트 역할 종류 및 그에 따른 권한 등
- 기타: 지시/승인 등 사람용 **UX**, 협업 시스템 등

조금씩 서로 다른 축 (예: 에이전트 런타임의 지속성을 위한 관측 지식 툴 정책)

하네스 엔지니어링의 산출물

- 안내/지식: 지식 검색/관리 시스템
- 실행/도구: 툴, 구조화된 런타임 계약
- 안전/통제: **sandbox, secret** 관리 등
- 관측: 재현, 디버깅, 환경 개선 등을 위함
- 검증: 일 잘 했는지 판정하는 기준 및 테스트 (**acceptance**)
- 지속성: **handoff, progress** 등 아티팩트 및 그에 기반한 업무 프로세스 등 정의
- 정책: 권한 종류, 에이전트 역할 종류 및 그에 따른 권한 등 **프리셋** 제공 가능
- 기타: 지시/승인 등 사람용 **UX**, 협업 시스템 등

이 중 그나마 범용으로 쓸 수 있는 건?

관련 제 조사 방향

- 환경/정책 등에 대한 점진적 스펙
 - 무엇을 어떻게 요청하고 응답받고 어떤 권한/예산으로 움직이는지를 정의하는 틀 설계
 - 주관 없이 그냥 써도 기본 프리셋 좋아서 기본적으로 잘 동작하게
 - 필요에 따라 크게 손보지 않아도 실용적으로 커스터마이징 가능하게
 - 주관 잔뜩 넣어서 뻥세게 커스터마이징해도 그 밑은 쓸 수 있어서 유용하게
- 에이전트 런타임
 - 업무 프로세스나 지식 시스템 등을 고려해서 런타임 계약, 톨, 컨텍스트 번들 등 설계

관련 제 조사 - 권한 모델

(X) 단순 read-only나 write가 아니라

(O) 어떤 표면에 어떤 수준의 권한이 있고 각 예산(시간, 토큰 등)은 얼마나 있는지

- Product Code - 그 하네스에서 만들려는 것
- Derived State - 빌드/검색 캐시 등
- Control Plane - 하네스 자체의 동작
- Secret Plane - API 키 등 민감정보
- Runtime/External - 외부 CI/CD, SaaS/IaaS, 지식/협업 시스템 등

관련 제 조사 - 더 근본적인 것들

- Data Semantics: 무엇을 사실/정책/힌트/증거 등으로 볼 것인가
- Execution Semantics: 무엇을 run/task/step/action/handoff 등으로 볼 것인가
- Authority Surface: 권한을 어디에서 자르는가
- Context Radius: file/module/project/external 등 어떤 범위의 맥락이 필요한가
- Outcome Family: 성공 외에도 무엇을 정상 결과로 볼 것인가 (실패 분류 등)
- Orchestration Slot: 단일, 순차(계획/구현/리뷰 등), 병렬 중 어디에 놓이는가

※ 제 관심에 따른 조사 결과 요약이라 다소 제 아이디어 쪽으로 치우쳐 있습니다.

그 아이디어는 환경/정책 등에 대한 점진적 스펙, 에이전트 런타임 관련

관련 제 조사 - 환경/정책 스펙 주요 고려사항

- Generic Query를 정본으로
 - Bootstrap, file-context, rationale 등은 프로필, primitive는 subject × need 질의
- 사실/정책/역할/힌트 등 분리
 - 안 하면 설명 근거 약해지고 병합 우선순위 모호하고 관리 잘못될 수 있음
- 구조화된 계약 사용
 - tool call/result, partial/refusal, request/response, recovery candidate, handoff artifact 등
- 권한 적절히 나누기
 - affordance grants, authority surface, target binding, runtime budget, control policy 등
- provider-facing lowered contract 등을 분리해 주관 적게 유지
- 실패도 설계된 흐름으로 다루는 프로세스 설계
- 등등

관련 제 조사 - 에이전트 런타임 주요 고려사항

- 일급 객체 취급할 것들:
 - Run, Task, Step
 - ActionRequest
 - ActionResult
 - Artifact
 - Handoff
 - Checkpoint, Resume, Fork
 - ApprovalRequest, EscalationRequest, ElicitationRequest
 - Attempt
 - Observation
 - Signal
 - 그 외에도 그냥 message
 - 등등

관련 제 조사 - 에이전트 런타임 주요 고려사항

- 일급 객체 취급할 것들: Run/Task/Step/ActionRequest/Artifact/Fork 등
- 실행 기록은 문자열 로그 대신 구조화된 transcript로 취급
 - message
 - tool_call
 - tool_call_output
 - approval_request / response
 - elicitation_request / response
 - refusal
 - artifact_ref
 - item.delta, item.done
 - system_error
 - 등등

관련 제 조사 - 에이전트 런타임 주요 고려사항

- 일급 객체 취급할 것들: Run/Task/Step/ActionRequest/Artifact/Fork 등
- 실행 기록은 문자열 로그 대신 구조화된 transcript로 취급
- 좋은 Context Engine
 - context assemble
 - compact
 - reset
 - handoff
 - inherit
 - dedupe
 - rehydrate
 - stale detection
 - 등등

관련 제 조사 - 에이전트 런타임 주요 고려사항

- 일급 객체 취급할 것들: Run/Task/Step/ActionRequest/Artifact/Fork 등
- 실행 기록은 문자열 로그 대신 구조화된 transcript로 취급
- 좋은 Context Engine
- sandbox-native
 - sandbox broker
 - tool gateway
 - session server
 - trace/replay store
- 기타
 - self-heal in derived-state
 - fallback to simpler retriever
 - quarantine broken override

관련 제 조사 - 기타 주요 고려사항

- Step별 목표 축 분리: outcome/process/style/efficiency/...
- **Acceptance** 평가/테스트
- 협업도 에이전트 우선으로 관리 - 온톨로지 스타일 구조화된 지식 베이스
- 지식 품질 관리: 지속적으로 측정하고 관리해야 함
 - stale object rate, citation coverage, conflicting active claims
 - unresolved candidate backlog, task-to-knowledge closure rate
 - retrieval recall/noise, duplicate read rate, compaction loss rate
 - approval fatigue, policy-caused failure vs agent-caused failure

관련 제 조사 - 정책 관련 고려사항

- 정책도 버전/배포/감사되는 소프트웨어나 지식처럼 지속적으로 관리해야 함
- policy representation
- versioning / migration
- rollout stage
- exception / override
- audit trail
- denied-action history
- defense quality evaluation
- preset packaging
- 등등

관련 제 조사 - 정책 관련 고려사항

- 정책도 버전/배포/감사되는 소프트웨어나 지식처럼 지속적으로 관리해야 함
- Rollout 프로세스
 - draft
 - pilot
 - limited/default-on
 - strict enforcement
 - drift detection / cleanup
- 측정 없이 강화하지 말고, 지식처럼 지속적으로 평가하고 관리해야 함
 - false positive
 - false negative
 - unblock latency
 - override frequency

마무리 - 결국 바뀐다

- 에이전트 런타임 예시 - 구조화된 출력 잘 하게 하기
 - 모델은 단순히 직관적으로 피드백도 받아가면서 자연스럽게 출력
 - 예: XML/JSX 스타일로 출력, 모델 외부 시스템에서 조립/피드백
 - 근데 모델 성능이 발전한다면, 외부 시스템에서 어디까지 조립하는 게 최적?
- 다른 예시 - **handoff** 남기기 스킬
 - 방법 알려주고 서브에이전트 검증 프로세스 만들고, ...
 - 모델이 그 패턴 자체를 잘 학습해서 안 알려줘도 알아서 잘 한다면?
 - 프롬프트 엔지니어링의 추론 유도처럼, 나중에는 필요 없을지도?

마무리 - 생각해볼 점 ~~(아님)~~

1. 에이전트가 충분히 똑똑하다면 하네스 같은 게 필요할까요?
 - 미래에는 AI 모델이 추상화된 코드가 아니라 기계어 바이너리를 바로 출력할 것도 같았던데...
2. 하네스 엔지니어링은, 왜 **하네스** 엔지니어링이라고 할까요?
 - 일단 사람한테 일 잘 하게 하는 업무 환경을 하네스라고 하면 기분 나쁘긴 할 듯 ㅋㅋㅋ
3. 그리고 하네스 엔지니어링을, 과연 모두가 해야 할까요?
 - 아직 없긴 한데, 그냥 누가 제대로 된 범용 SaaS 하나 만들어주면...

사실 그냥 여러분의 의견이 궁금해서 적어봤어요

마무리 - 결론

1. 절찬 구직중, 많관부!
2. 연구 관심/질문 있거나 친해지고 싶거나 하신 분들 부담 없이 연락 주세요!
3. 다들 즐일하세요!

GitHub: <https://github.com/mjy9088>

이력서: <https://bit.ly/mjy-resume>

연락처: 카카오톡 또는 이력서의 전화번호/이메일